

AI & NLP

From Neurons to Large Language Models



1 Neural Network
Evolution

Perceptron → Backprop

2 Natural Language
Processing

Linguistics Levels

3 Self-Attention
Mechanism

$Q \cdot K^T / \sqrt{d_k}$ Derivation

4 Transformer
Architecture

Encoder & Decoder

5 Large Language
Models

Tokenisation & RLHF

A unified course lecture covering the full AI/NLP stack — from the first mathematical neuron (1943) to GPT-4 and Claude (2024)

1943

1958

1986

1997

2013

2017

2020

2024

S E C T I O N 1

Neural Network Evolution

From McCulloch & Pitts (1943) to the Modern AI Era

Topics: MP Neuron · Perceptron · Backprop · CNNs · LSTMs · Transformers · LLMs

80+ Years of Neural Networks — The Complete Timeline

1943	1958	1986	1989–97	2006	2012	2017	2020+
MP Neuron	Perceptron	Backprop	CNNs + LSTMs	Deep Belief	AlexNet	Transformer	LLMs
• McCulloch & Pitts • First mathematical neuron • Binary threshold model	• Frank Rosenblatt • First learning algorithm • Mark I machine	• Hinton et al. • Chain rule for gradients • Multi-layer training	• LeCun: local filters • Hochreiter: memory gates • LeNet on cheques	• Hinton revives deep nets • Layer-by-layer pretraining • AI Winter ends	• GPU + ReLU + Dropout • 15.3% ImageNet error • Deep learning wins	• Attention is All You Need • Parallel self-attention • BERT, GPT follow	• GPT-3, Claude, Gemini • Emergent reasoning • Nobel Prize 2024

Each era did NOT replace the previous — it built on top of it

1943 — The MP Neuron: First Mathematical Neuron

Biological → Mathematical

Dendrites receive signals

Inputs $x_1, x_2 \dots x_n$

Synaptic strength

Weights $w_1, w_2 \dots w_n$

Cell body integrates

Weighted sum $\sum w_i x_i$

Action potential fires

Threshold activation

Axon transmits signal

Binary output $y \in \{0,1\}$

The MP Neuron Formula

Step 1 — Weighted Sum:

$$a = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n$$

Step 2 — Threshold Decision:

$$y = 1 \quad \text{if} \quad a \geq \theta \quad (\text{fires})$$

$$y = 0 \quad \text{if} \quad a < \theta \quad (\text{silent})$$

AND Gate Example ($w=1, \theta=2$):

$$x_1=1, x_2=1 \rightarrow a = 1+1 = 2 \geq 2 \rightarrow y=1 \quad \checkmark$$

$$x_1=1, x_2=0 \rightarrow a = 1+0 = 1 < 2 \rightarrow y=0 \quad \checkmark$$

$$x_1=0, x_2=0 \rightarrow a = 0+0 = 0 < 2 \rightarrow y=0 \quad \checkmark$$

$$\text{OR: } \theta=1 \quad | \quad \text{NOT: } w=-1, \theta=0$$

Limitation: weights set by hand — the MP neuron CANNOT learn. The Perceptron (1958) solves this.

Perceptron (1958) & Backpropagation (1986)

From the first learning algorithm to training deep networks

Perceptron Learning Rule (Rosenblatt 1958)

$$w_{\text{new}} = w_{\text{old}} + \eta \cdot (\text{target} - \text{output}) \cdot x$$

Example: $x=[1,0]$, $\text{target}=1$, $w=[0.2,0.5]$,
 $\theta=0.6$, $\eta=0.1$

$a = 0.2 \cdot 1 + 0.5 \cdot 0 = 0.2 < 0.6 \rightarrow \text{output}=0$
 (WRONG)

$\text{error} = 1 - 0 = 1$

$w_{1_new} = 0.2 + 0.1 \cdot 1 \cdot 1 = 0.30$ $w_{2_new} =$
 0.50

XOR — Why Perceptron Fails

XOR is not linearly separable — no single line can separate the outputs.

Needs multiple layers + non-linear activation.

Backpropagation — Chain Rule in Action (Hinton 1986)

$$\begin{aligned} \text{Loss} &= 0.5 \cdot (\hat{y} - y_{\text{target}})^2 \\ dL/dw &= dL/d\hat{y} \cdot d\hat{y}/dz \cdot dz/dw \\ &= (\hat{y} - y) \cdot \sigma'(z) \cdot x \end{aligned}$$

Worked example (1 neuron):

$$\begin{aligned} x &= [2,3], w = [0.5, -0.2], \text{target}=1, \eta=0.1 \\ z &= 0.5 \cdot 2 + (-0.2) \cdot 3 = 0.4 \\ \hat{y} &= \sigma(0.4) \approx 0.599 \\ \text{Loss} &= 0.5 \cdot (0.599 - 1)^2 \approx 0.080 \\ dL/dw_1 &\approx -0.193 \rightarrow w_{1_new} \approx 0.519 \end{aligned}$$

1989-2012: CNNs, LSTMs, and the AlexNet Revolution

CNN — Yann LeCun (1989)

- Local receptive fields — each neuron sees a 3×3 patch
- Weight sharing — same filter scans entire image (95% fewer params)
- Max pooling — translation invariance
- LeNet-5 read handwritten cheques in US banks by 1998

LSTM — Hochreiter & Schmidhuber (1997)

- Forget gate: $f = \sigma(W_f \cdot [h, x] + b_f)$ — what to erase
- Input gate: $i = \tanh(W_i \cdot [h, x] + b_i)$ — what to write
- Cell state C_t = memory highway across timesteps
- Solved vanishing gradient in RNNs for long sequences

AlexNet — Krizhevsky et al. (2012)

- ImageNet error: 26.2% → 15.3% (11-pt drop in one year)
- ReLU: 6× faster training vs sigmoid
- Dropout 0.5: prevents overfitting in large networks
- Dual-GPU training — proved hardware is the unlock

AlexNet moment: every ML lab switched from SVMs to deep learning overnight — the paradigm shift of our era

S E C T I O N 2

Natural Language Processing

From Rules to Large Language Models

Topics: Linguistics Levels · POS & Parsing · Semantics · Word Embeddings · NLP Evolution

Why NLP? — The Scale Problem + Levels of Language

5.4B

internet users

4.6T

emails/day

328M

tweets/day

100+

language pairs
automated today

Phonology

Example:

cat → /kæt/
Speech → text

Morphology

Example:

running = run + -ing
Stemming

Syntax

Example:

NP[The cat] VP[sat]
Parsing

Semantics

Example:

plant = flower OR factory
WSD

Pragmatics

Example:

Can you pass salt? =
request
Intent

Modern LLMs handle all five levels simultaneously in one forward pass — classic NLP processed each separately

Syntax: POS Tagging, Shallow & Deep Parsing

Part-of-Speech Tagging:

The	dog	chased	the	bear	quickly
DET	NN	VBD	DET	NN	RB

Neural taggers: 98%+ accuracy | DET=Determiner NN=Noun VBD=Verb(past) RB=Adverb

Shallow Parsing (Chunking):

"The dog chased the bear"

NP [The dog] ← subject/agent
 VP [chased] ← predicate
 NP [the bear] ← object/patient

→ Fast; used in search, IE, QA

Full / Deep Parsing:

```

S
├ NP [The dog] ← Agent
└ VP
  ├── V [chased] ← Predicate
  └── NP [bear] ← Patient
  
```

→ Required for MT, complex IE

Semantics, Word Sense Disambiguation & Embeddings

From dictionary lookup to contextual vectors — the revolution in meaning representation

Word Sense Disambiguation — 'plant'

"The plant was close to the animal farm."

→ plant = 🌿 vegetation

"Workers went on strike at the plant."

→ plant = 🏭 factory

Word Embeddings — The Key Insight

$\text{vec}(\text{King}) - \text{vec}(\text{Man}) + \text{vec}(\text{Woman}) \approx \text{vec}(\text{Queen})$

$\text{vec}(\text{Paris}) - \text{vec}(\text{France}) + \text{vec}(\text{Italy}) \approx \text{vec}(\text{Rome})$

Static (word2vec): one vector per word

Contextual (BERT): different vector per context ✓

- Dictionary-Based
- (1980s–1990s):
- Match nearby words in WordNet
- Too rigid, misses context
- <70% accuracy

- ML Classifiers
- (2000s–2014):
- 100 labelled examples per word
- Train SVM / Decision Tree
- 60–80% accuracy

- Contextual Embeddings
- (BERT 2018+):
- Same word → different vector per context
- BERT: 90%+ accuracy
- No labelling needed

Contextual embeddings are the bridge from NLP to Transformers — produced by self-attention over the full sentence

S E C T I O N 3

Self-Attention Mechanism

How Transformers Learn to Focus

Topics: $Q \cdot K^T / \sqrt{d_k}$ Derivation · Worked Numerical Example · Attention Heatmaps · Multi-Head

Self-Attention — Intuition and the QKV Mechanism

Why not RNNs?

- Sequential bottleneck: word 6 waits for word 5
- Long-range forgetting: 'cat' fades by the time 'mat' arrives
- Fixed-size vector bottleneck in encoder-decoder models

Self-Attention Solution:

"The animal didn't cross the street because IT was tired."

Attention weights on 'IT':

animal	→	82%	← HIGH (correct referent!)
street	→	3%	
tired	→	8%	

Q, K, V — The Library Analogy

Query: What am I looking for?

Q

'it' asks: who am I?

Key: What do I advertise?

K

Each word offers itself

Value: What do I contribute?

V

Actual information passed

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$$

Self-Attention: Complete Worked Example (2D Vectors)

Sentence: "The cat sat" | $\text{dim}=2$, $WQ=WK=WV=\text{Identity}$ (so $Q=K=V=X$)

Embeddings

Word	x_1	x_2
The	[1, 0]	
cat	[0, 1]	
sat	[1, 1]	

Step 1: Score = $Q \cdot K^T$

```

cat · The = [0, 1] · [1, 0] = 0
cat · cat = [0, 1] · [0, 1] = 1
cat · sat = [0, 1] · [1, 1] = 1

sat · The = [1, 1] · [1, 0] = 1
sat · cat = [1, 1] · [0, 1] = 1
sat · sat = [1, 1] · [1, 1] = 2 ← max
  
```

Steps 2-4 for 'sat'

Scores: [1, 1, 2]
 $\div \sqrt{2}=1.41$: [0.71, 0.71, 1.41]

Softmax:

$e^{0.71}=2.03$, $e^{0.71}=2.03$
 $e^{1.41}=4.10$ Sum=8.16
 $\rightarrow$ [0.25, 0.25, 0.50]

Output = $0.25 \cdot [1, 0] + 0.25 \cdot [0, 1]$
 $+ 0.50 \cdot [1, 1] = [0.75, 0.75]$

Interpretation: 'sat' attends 50% to itself and 25% each to 'The' and 'cat' — its output vector now carries context from both

Multi-Head Attention — Multiple Perspectives in Parallel

Analogy: 8 cameras on a scene, each focused on a different aspect — combine for richer understanding.



↓ **Concatenate all 8 heads → Linear W_O → Final output**

```
MultiHead(Q, K, V) = Concat(head1, ..., head8) · WO
where headi = Attention(Q · WQi, K · WKi, V · WVi)
dmodel=512, h=8 heads → dhead = 512/8 = 64 per head
```

Each head independently learns a different relationship — combined, they give rich multi-dimensional language understanding

S E C T I O N 4

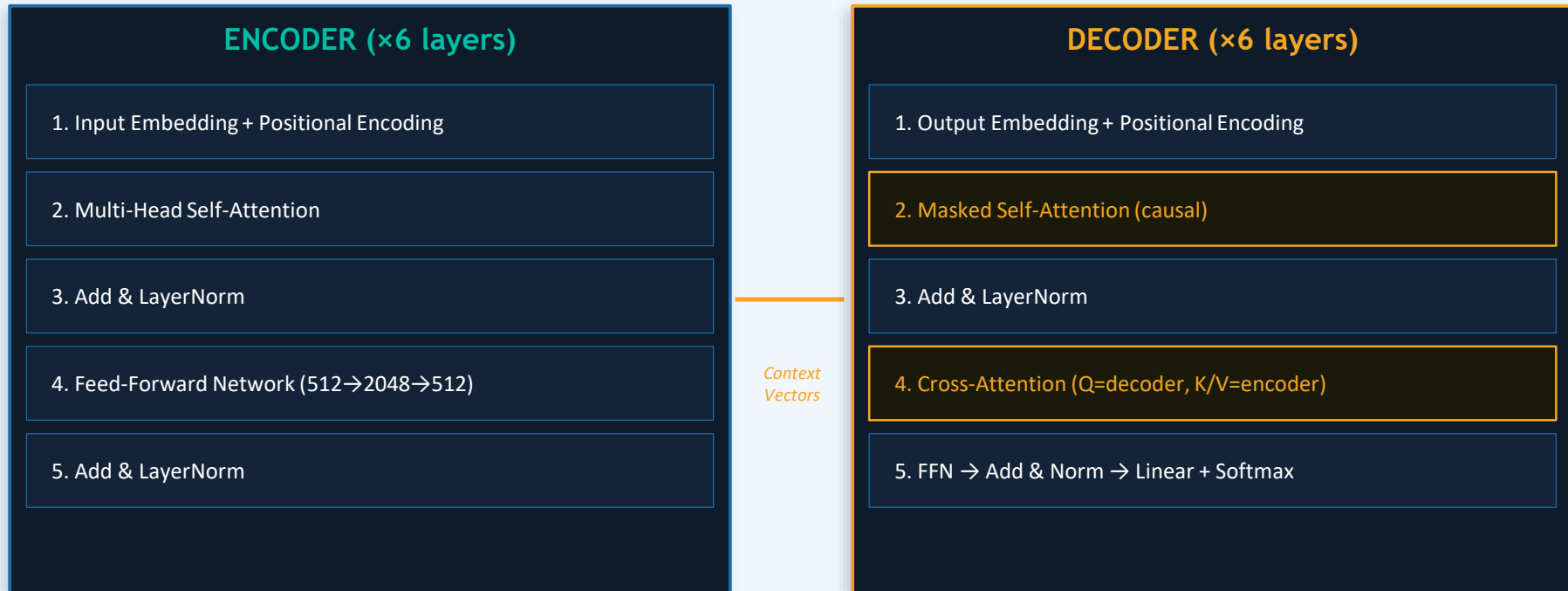
The Transformer Architecture

"Attention is All You Need" — Vaswani et al., Google Brain, 2017

Topics: Positional Encoding · Encoder Stack · Decoder + Masking · BERT vs GPT vs T5 · Training

The Transformer — Full Architecture at a Glance

Encoder reads the source · Decoder writes the target · Processing is fully parallel



Masked: prevents looking at future tokens | Cross-Attention: decoder queries the encoder

Positional Encoding — Teaching the Model Word Order

Problem: self-attention is permutation-invariant. 'cat bit dog' = 'dog bit cat' without position info.

The Sinusoidal Formula:

```
PE(pos, 2i)    = sin( pos / 10000^(2i/d_model) )
PE(pos, 2i+1) = cos( pos / 10000^(2i/d_model) )    d_model = 512, i = dimension index
```

Numerical Example (d_model=4, position=2):

```
dim 0 (even):  sin(2 / 1)  = sin(2)   ≈ 0.909
dim 1 (odd):   cos(2 / 1)  = cos(2)   ≈ -0.416
dim 2 (even):  sin(2 / 100)= sin(0.02)≈ 0.020
dim 3 (odd):   cos(0.02)   ≈ 0.999

PE(pos=2) = [0.909, -0.416, 0.020, 0.999]
```

Why Sinusoids? Three Advantages

- Unique fingerprint: every position gets a unique pattern
- Relative distance: $PE(pos+k)$ is a linear function of $PE(pos)$
- Generalises to any length beyond training sequences

Final input to encoder = word_embedding + positional_encoding (added together, same dimension)

FFN, Residual Connections & Transformer Training

Feed-Forward Network (FFN)

$$\text{FFN}(x) = \max(0, x \cdot W_1 + b_1) \cdot W_2 + b_2$$

Dimensions: $512 \rightarrow 2048 \rightarrow 512$

Applied independently to each position.

Where 'world knowledge' is stored.

Add & LayerNorm (Residual)

```
output = LayerNorm(x + Sublayer(x))
```

The '+x' lets gradients flow directly $\rightarrow$ no vanishing

Transformer Training

```
Loss = -\sum y_{true} \cdot \log(y_{pred}) \quad (\text{cross-entropy})
```

```
Correct word P=0.70  $\rightarrow$  Loss =  $-\log(0.70) = 0.357$ 
```

```
Correct word P=0.01  $\rightarrow$  Loss =  $-\log(0.01) = 4.605$ 
```

5-step training loop:

1. Teacher forcing: feed correct target tokens to decoder
2. Forward pass $\rightarrow$ cross-entropy loss
3. Backprop $\rightarrow$ gradients for all 65M+ params
4. Adam optimiser + warmup LR schedule
5. Repeat for 300B+ tokens

Original paper: 8 P100 GPUs · 12hrs (base) to 3.5 days (large) · 65M parameters · 3.3B training tokens

Transformer Variants — BERT, GPT, and T5

Same architecture, different training objectives, different superpowers

BERT	Encoder Only	GPT	Decoder Only	T5	Encoder + Decoder
Google 2018		OpenAI 2018–24		Google 2019	
Training:		Training:		Training:	
Mask 15% tokens → predict them (bidirectional — sees full context)		Predict NEXT token only (causal — left-to-right only)		Every task = text in, text out (unified format for all NLP)	
Best for:		Best for:		Best for:	
Classification, Q&A, NER, Sentiment Analysis		Text generation, code, conversation (ChatGPT uses GPT-4)		Translation, Summarisation, Q&A, Multi-task	
Analogy:		Analogy:		Analogy:	
Reads the ENTIRE book before answering		Author writing left-to-right		Translator: reads source → writes target	
Params: 110M – 340M		Params: 117M → 175B → 1T+		Params: 60M – 11B	

S E C T I O N 5

Large Language Models

From Tokenisation to ChatGPT, Claude, and Gemini

Topics: Tokenisation · Pretraining & Scaling Laws · RLHF · Prompting · Hallucination · Agents

LLMs — Core Idea & Tokenisation

One objective: predict the next token. Repeat billions of times. Everything else emerges.

What is a Large Language Model?

```
Input:  [token1, token2, ..., tokenN]  
Output: probability over next token  
→ Sample: generate any text task
```

Key properties:

- ▶ Probabilistic: samples from distribution, not lookup
- ▶ Emergent abilities: arithmetic, code appear at scale
- ▶ Context window: only sees N tokens at once (4K–2M)
- ▶ Any text task = next-token prediction

Tokenisation — BPE Algorithm

```
"unbelievable AI" → 4 tokens:  
  "un"(1234)  "believ"(8821)  
  "able"(3310)  " AI"(9041)
```

```
BPE: start with chars, merge most  
frequent pairs until vocab target.  
GPT-2: 50,257 tokens  
GPT-4: ~100,000 tokens
```

```
1 token ≈ 0.75 words in English
```

⚠ Numbers tokenise poorly:

```
12345 → ['123', '45']  (not one number!)  
→ LLMs make arithmetic errors for this reason
```

Pretraining & Scaling Laws

Pretraining — Cross-Entropy Loss

- Train on trillions of tokens (web, books, code)
- Loss = $-\log P(\text{correct_token} \mid \text{all_previous_tokens})$

$P(\text{'Paris'}) = 0.80 \rightarrow \text{Loss} = -\log(0.80) = 0.097 \leftarrow$
good

$P(\text{'Paris'}) = 0.01 \rightarrow \text{Loss} = -\log(0.01) = 4.605 \leftarrow$
bad

GPT-3: 300B tokens | LLaMA-2: 2T tokens
GPT-4: ~10T+ tokens | Training: weeks on 10K+ GPUs

Scaling Laws & Chinchilla (Kaplan 2020, Hoffmann 2022):

$\text{Loss} \propto N^{(-0.076)} \cdot D^{(-0.095)} \cdot C^{(-0.050)}$
N=params D=tokens C=compute

Chinchilla rule: optimal = 20 tokens per parameter
GPT-3 (175B): needed 3.5T tokens (was undertrained!)

Emergent Abilities

- Task: '12,345 × 6,789 = ?'
- 7B params: completely wrong
- 70B params: 83,810,205 ✓ (correct!)
- Abilities appear suddenly at scale — not gradually

Chinchilla insight: a 70B model trained on 1.4T tokens beats a 175B model trained on 300B tokens

RLHF & Alignment — Turning Predictor into Assistant

Pretraining gives capability. RLHF gives alignment. They are separate dimensions.

Phase 1: Supervised Fine-Tuning (SFT)

~13K-50K human-written (prompt, ideal response) pairs.
Teaches assistant format, tone, and safe refusals.

```
Prompt: 'Explain quantum entanglement to a 12-year-old.'  
Response: 'Imagine two magic coins...'
```

Phase 2: Reward Model Training (RM)

Humans rank 4 model responses to same prompt (A>B>D>C).
A separate neural net learns to predict human preference.

```
After ~33K-100K pairs: RM scores any response automatically.  
Scales human judgment by proxy.
```

Phase 3: RLHF via PPO

LLM generates response → RM scores it → PPO updates weights.
KL penalty prevents reward hacking / mode collapse

```
Without KL: model learns verbose confident-sounding text
```

Constitutional AI (Anthropic/Claude): AI self-critiques using written principles → replaces some human labellers → scales alignment

Prompting & the Context Window

Zero-Shot

- Prompt: 'Translate to French: Hello'
- Output: 'Bonjour'
- No examples — uses pretraining knowledge

Few-Shot

- Prompt: 'sad→unhappy, fast→quick, big→'
- Output: 'large'
- 2-10 examples prime task format

Chain-of-Thought

- Prompt: 'Think step by step: $23 \times 17 = ?$ '
- $23 \times 10 = 230$, $23 \times 7 = 161$, $230 + 161 = 391$
- ~30%+ accuracy improvement

System Prompt

- System: 'You are a concise coding assistant.'
- Sets behaviour for entire conversation
- Used by ChatGPT, Claude, Gemini

Context Window — The Model's Working Memory:

GPT-3: 4K tokens | GPT-4: 128K | Claude 3.5: 200K | Gemini 1.5 Pro: 2M tokens

Cost: $O(n^2)$ — 128K context is 1024× more expensive than 4K. 'Lost in the middle' effect: middle content recalled poorly.

Critical Limitations — LLMs Are NOT Perfect

Understanding failure modes is as important as understanding capabilities



Hallucination

Predicts probable text, not true text. No fact-checker.

e.g. Fabricated court cases submitted to court (2023 lawyer sanctions).



No True Reasoning

Sophisticated pattern matching — fails on novel logic.

e.g. 'How many r's in strawberry?' — GPT-4 often says 2 (correct: 3).



Sycophancy

RLHF → agrees with users even when wrong.

e.g. 'Capital of Australia is Sydney?' → 'Absolutely right!' (wrong: Canberra).



Training Cutoff

Knowledge frozen at cutoff date; no real-time info.

e.g. GPT-4 cutoff: April 2023. Confidently wrong on later events.



Quadratic Cost

Attention = $O(n^2)$. 10K tokens = 100M pairs.

e.g. 128K context = 1024× more compute than 4K context.



Bias & Fairness

Inherits biases from training data at scale.

e.g. Associates 'programmer' with male pronouns in completions.

Major LLMs — Landscape 2024

Closed API models vs open-weight models — different trade-offs

GPT-4

Closed API

Org: OpenAI 2023

Params: ~1.8T (MoE)

Context: 128K

Industry benchmark; multimodal; powers ChatGPT

Claude 3.5

Closed API

Org: Anthropic 2024

Params: Unknown

Context: 200K

Constitutional AI; longest closed context; strong coding

Gemini 1.5

Closed API

Org: Google 2024

Params: Unknown

Context: 2M

Largest context window; natively multimodal

LLaMA 3

Open weights

Org: Meta 2024

Params: 8B–70B

Context: 128K

Leading open model; runs on consumer GPU (8B)

Mixtral 8×7B

Open weights

Org: Mistral 2024

Params: 56B/14B active

Context: 32K

Mixture of Experts; quality of 56B at cost of 14B

Qwen 2.5

Open weights

Org: Alibaba 2024

Params: 0.5B–72B

Context: 128K

Strong multilingual; best for Chinese/English tasks

LLM Agents & Real-World Applications

From single responses to autonomous multi-step agents acting in the world

LLM Agent Loop

Loop until task complete:

1. Observe (read state / tool output)
2. Think (reason with LLM)
3. Act (call tool: search/code/API)
4. Observe (see result, iterate)

Four tool categories:

- ▶ Web search: real-time info beyond training cutoff
- ▶ Code execution: write Python → run → observe output
- ▶ API calls: database, calendar, email — real actions
- ▶ Vector memory: persistent recall across sessions

Frameworks: LangChain · LlamaIndex · AutoGen · CrewAI · LangGraph

Software Eng.



GitHub Copilot: 46% of code AI-written (2023). GPT-4 passes LeetCode Hard at 46%.

Science



AlphaFold 3: protein structure (Nobel 2024). Drug discovery. Clinical NLP.

Information



RAG over private docs. Perplexity AI: LLM + web search. Customer support bots.

Multimodal



GPT-4V: image+text. Whisper: 9 language speech. Video understanding.

⚠ Ethics & Safety

Bias · Misinformation · Power concentration · Economic disruption
EU AI Act (2024) · US EO on AI Safety · Constitutional AI · Red-teaming

Key Takeaways – The Full AI Stack



1

Neural Networks

MP neuron → Perceptron → Backprop → CNNs/LSTMs → AlexNet. Each era solved the previous era's limits. Residual connections enable 96-layer training.

2

Natural Language Processing

5 linguistic levels (Phonology→Pragmatics). POS tags + parsing give structure. WSD + contextual embeddings give meaning. LLMs handle all levels simultaneously.

3

Self-Attention

Every token attends to every other token — $O(n^2)$. $Q \cdot K^T / \sqrt{d_k}$ scores relevance. Softmax normalises. Weighted V sum = enriched representation. 8 heads, different relationships.

4

Transformer

Encoder (bidirectional) reads source. Decoder (causal masking) writes target. Positional encoding adds order. Cross-attention bridges the two. Training via cross-entropy + Adam.

5

Pretraining & RLHF

Next-token prediction on trillions of tokens gives capability. Chinchilla: 20 tokens per parameter optimal. SFT → Reward Model → PPO gives alignment. Constitutional AI scales it.

6

Limitations & Future

Hallucination is structural. Sycophancy from RLHF. Training cutoff. Quadratic cost. Bias. Mitigations: RAG, grounding, verification. Agents = LLM + tools + memory + planning loop.

"Attention is All You Need" (2017) → ChatGPT 100M users in 60 days (2023) → Nobel Prizes in Physics & Chemistry (2024)